Gestión de datos Amazon Redshift

A. Gestión de una base de datos en Amazon Redshift.

1) Creamos la base de datos “qa”

```
CREATE DATABASE qa;
ALTER DATABASE qa OWNER TO awsuser;
ALTER DATABASE qa CONNECTION LIMIT 50;

SELECT datname, datdba, datconnlimit
FROM pg_database_info
WHERE datname= 'qa';
```

abase_info 1 | Estadísticas 1

datname	datdba	datconnlimit
qa	100	50

2) Cambiamos el nombre y las conexiones

```
/* Cambiar el nombre de la base de datos */
ALTER DATABASE qa RENAME TO prod;
/* Cambiar el límite de conexiones de la base de datos */
ALTER DATABASE prod CONNECTION LIMIT 100;

SELECT datname, datdba, datconnlimit
FROM pg_database_info
WHERE datname = 'qa';
```

database_info 1 | pg_database_info 1 (2) | Estadísticas 1

datname	datdba	datconnlimit

3) Comprobamos las conexiones

```
SELECT procpid, usename, datname
 FROM pg_stat_activity
 WHERE datname = 'prod';
```

tat_activity 1

123 procpid	A-Z usename	A-Z datname

4) Eliminamos la base de datos

```
DROP DATABASE prod;
```

Estadísticas 1

Name	Value
Updated Rows	0
Execute time	0,369s
Start time	Wed Mar 11 10:20:25 CET 2026
Finish time	Wed Mar 11 10:20:26 CET 2026
Query	DROP DATABASE prod

B. Gestión de un esquema en una base de datos.

1) Creamos los grupos y usuario

```
CREATE GROUP finance_grp;
CREATE GROUP audit_grp;
CREATE USER finance_admin_usr WITH PASSWORD 'Contraseña1';
```

2) Creamos el esquema y lo gestionamos

```
CREATE SCHEMA finance AUTHORIZATION finance_admin_usr QUOTA 1 TB;

GRANT USAGE ON SCHEMA finance TO GROUP finance_grp;
GRANT USAGE ON SCHEMA finance TO GROUP audit_grp;
GRANT ALL ON SCHEMA finance TO GROUP finance_grp;
GRANT SELECT ON ALL TABLES IN SCHEMA finance TO GROUP audit_grp;
```

3) Verificamos la creación

```
select nspname as schema, usename as owner
from pg_namespace, pg_user
where pg_namespace.nspowner = pg_user.usesysid
and pg_namespace.nspname ='finance';
```

g_namespace(+) 1 | foo 1 (2) | foo 1 (3) | Estadísticas 1

lect nspname as schema, usename as owner from pg_namespace, pg | Data filter is not supported

	schema	owner
1	finance	finance_admin_usr

4) Creación de la tabla foo.

```
CREATE TABLE finance.foo (bar int);

select * from finance.foo;

set search_path to '$user', finance, public;

select * from foo;
```

C. Gestión de tablas en una base de datos.

1) Creamos la tabla customer.

```
CREATE TABLE finance.customer
(
 customer_number INTEGER,
 first_name VARCHAR(50),
 last_name VARCHAR(50),
 date_of_birth DATE
);

insert into finance.customer values
(1, 'foo', 'bar', '1980-01-01'),
(2, 'john', 'smith', '1990-12-01'),
(3, 'spock', 'spock', '1970-12-01'),
(4, 'scotty', 'scotty', '1975-02-01'),
(5, 'seven', 'of nine', '1990-04-01'),
(6, 'kathryn', 'janeway', '1995-07-01'),
(7, 'tuvok', 'tuvok', '1960-06-10'),
(8, 'john', 'smith', '1965-12-01'),
(9, 'The Doctor', 'The Doctor', '1979-12-01'),
(10, 'B Elana', 'Torres', '2000-08-01');

select "schema", table_id, "table", encoded, diststyle, sortkey1, size, tbl_rows
from svv_Table_info
where "table" = 'customer'
and "schema" = 'finance';
```

tados 1 | Estadísticas 1

schema	table_id	table	encoded	diststyle	sortkey1	size	tbl_rows
finance	108960	customer	Y, AUTO(ENCODE)	AUTO(ALL)	AUTO(SORTKEY)	7	10

2) Consulta para crear tabla temporal.

```
select *
 from finance.customer
 where extract(year from date_of_birth) < 1980;
```

ltados 1 | customer 1 (2) | #customer 1 (3) | Estadísticas 1

123 customer_number	A-Z first_name	A-Z last_name	date_of_birth
3	spock	spock	1970-12-01
4	scotty	scotty	1975-02-01
7	tuvok	tuvok	1960-06-10
8	john	smith	1965-12-01
9	The Doctor	The Doctor	1979-12-01

3) Creamos tabla temporal.

```
create table finance.customer_dob_1980 as
 select *
 from finance.customer
 where extract(year from date_of_birth) = 1980;

 create temporary table #customer(custid integer IDENTITY(1,1), customer_number integer);
 insert into #customer (customer_number) values(1);
 select * from #customer;
```

dos 1 | customer 1 (2) | #customer 1 (3) | Estadísticas 1

123 custid	123 customer_number
1	1

Valor

D. Gestión de vistas en una base de datos.

1) Creamos las vistas

```
CREATE VIEW finance.customer_vw
 AS
 SELECT customer_number,
      first_name,
      last_name,
      EXTRACT(year FROM date_of_birth) AS year_of_birth
 FROM finance.customer;

CREATE VIEW finance.customer_lbvw
 AS
 SELECT customer_number,
     first_name,
     last_name,
     EXTRACT(year FROM date_of_birth) AS year_of_birth
 FROM finance.customer
 with no schema binding;

SELECT table_schema as schema_name,
     table_name as view_name,
     view_definition
 FROM information_schema.views
 WHERE table_schema not in ('information_schema', 'pg_catalog')
 ORDER by schema_name,
     view_name
```

iews 1 | Estadísticas 1

	schema_name	view_name	view_definition
1	finance	customer_lbvw	CREATE VIEW finance.customer_lbvw¶AS¶SELECT customer_nu
2	finance	customer_vw	SELECT customer.customer_number, customer.first_name, cus
3	pg_auto_copy	copy_job_detail	[NULL]

2) Observamos customer_vw.

```
SELECT * from finance.customer_vw limit 5;
```

customer 1 (2) | Estadísticas 1

123 customer_number	A-Z first_name	A-Z last_name	123 year_of_birth
1	foo	bar	1.980
2	john	smith	1.990
3	spock	spock	1.970
4	scotty	scotty	1.975
5	seven	of nine	1.990

3) La eliminamos

```
DROP VIEW finance.customer_vw;
```

E. Gestión de vistas materializadas en una base de dato.

1) Creamos una vista materializada

```
CREATE MATERIALIZED VIEW finance.customer_agg_mv
AUTO REFRESH YES
AS
SELECT
    EXTRACT(year FROM date_of_birth) AS year_of_birth,
    count(1) customer_cnt
FROM finance.customer
group by EXTRACT(year FROM date_of_birth);

select * from finance.customer_agg_mv limit 5;
```

bl_customer_agg_mv_0 1 | Estadísticas 1

Enter a SQL expression to filter results (use Ctrl+Space)

123 year_of_birth	123 customer_cnt
1.980	1
1.970	1
1.975	1
1.960	1
1.965	1

2) La observamos

```
select * from SVV_MV_INFO where name='customer_agg_mv';
```

database_name	schema_name	user_name	name	is_stale
dev	finance	awsuser	customer_agg_mv	f

3) Insertamos datos y comprobamos

```
insert into finance.customer values
(11, 'mark', 'bar', '1980-02-01'),
(12, 'pete', 'smith', '1990-2-01'),
(13, 'woofy', 'spock', '1980-11-01'),
(14, 'woofy jr', 'scotty', '1975-03-01'),
(15, 'eleven', 'of nine', '1990-07-01');

select name,is_stale,state from SVV_MV_INFO where name='customer_agg_mv';
```

name	is_stale	state
customer_agg_mv	t	1

4) Refrescamos y comprobamos

```
REFRESH MATERIALIZED VIEW finance.customer_agg_mv;

select name,is_stale, state from SVV_MV_INFO where name='customer_agg_mv';
```

name	is_stale	state
customer_agg_mv	t	1

F. Gestión de procedimientos almacenados en una base de datos.

1) Creamos el esquema y el procedimiento

```
Create schema cookbook;

create or replace procedure sp_cookbook(indate in date, records_out INOUT
refcursor) as
$$
declare
 integer_var int;
begin
 RAISE INFO 'running first cookbook storedprocedure on date %', indate;
 drop table if exists cookbook.cookbook_tbl;
 create table cookbook.cookbook_tbl
 (recipe_name varchar(50),
 recipe_date date
 );
 insert into cookbook.cookbook_tbl values('stored procedure', indate);
 GET DIAGNOSTICS integer_var := ROW_COUNT;
 RAISE INFO 'rows inserted into cookbook_tbl = %', integer_var;
 OPEN records_out FOR SELECT * FROM cookbook.cookbook_tbl;
END;
$$ LANGUAGE plpgsql;
```

2) Llamamos al procedimiento

```
call sp_cookbook(current_date, 'inputcursor');
```

dos 1 | Estadísticas 1

records_out

inputcursor

3) Mostramos la construcción del procedimiento

```
SHOW PROCEDURE sp_cookbook(indate in date, records_out INOUT refcursor);
```

ltados 1 | Resultados 1 (2) | Estadísticas 1

AZ Stored Procedure Definition

```
CREATE OR REPLACE PROCEDURE public.sp_cookbook(indate date, INOUT records_out refcursor)¶
```

4) Creamos otro procedimiento

```
create or replace procedure public.sp_self_service(tblName in varchar(60)) as
$$
begin
 RAISE INFO 'running sp_self_service to drop table %', tblName;
 execute 'drop table if exists cookbook.' || tblName;
 RAISE INFO 'table dropped %', tblName;
END;
$$ LANGUAGE plpgsql
SECURITY DEFINER;
```

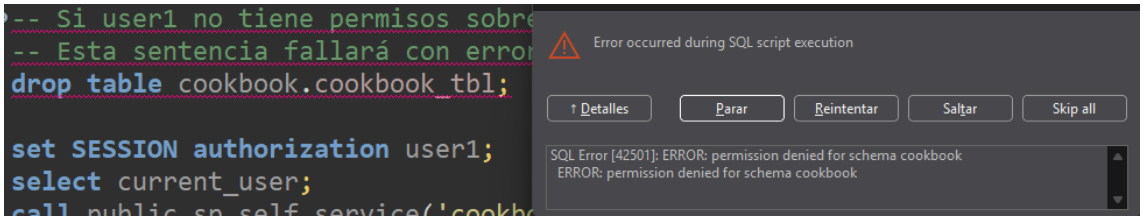
5) Creamos usuario y ejecutamos

```
create user user1 with password 'Cookbook1';
-- Otorga a user1 permiso para ejecutar el procedimiento sp_self_service.
grant execute on procedure public.sp_self_service(tblName in varchar(60)) to
user1;
-- Cambia el usuario activo de la sesión a user1.
-- A partir de aquí, todo se ejecuta como si fueras user1.
set SESSION authorization user1;
select current_user;
-- Si user1 no tiene permisos sobre esa tabla,
-- Esta sentencia fallará con error de permisos.
drop table cookbook.cookbook_tbl;

set SESSION authorization user1;
select current_user;
call public.sp_self_service('cookbook_tbl');
```

ados 1 | Resultados 1 (2) | Resultados 1 (3) | Resultados 1 (4) | Estadísticas 1

AZ current_user

user1

6) El drop table nos da error puesto que no tenemos permisos